

Demand-Driven CJ Enrichment

litellm microservices · 2026-07-13 · solves the shallow-catalog orderability problem (8,084 of 8,176 CJ goods browsable but not orderable) · companion to [doc/cj-pipeline-architecture-2026-07-13.pdf](#)

1 · Problem

The CJ catalog sync deliberately promotes every fetched product as a **browsable shallow goods**: one placeholder SKU with no `cj_vid`, an estimated "from" price, and default stock. Orderability requires **enrichment** — fetching the product's real variants from CJ (1 detail call + 1 inventory call per variant, globally paced at ~1 QPS) — which ran as a nightly batch of 20, oldest-first. Consequences:

- Only **92 of 8,176** CJ goods were orderable; customers hit a 422 at submit on everything else ("*CJ product ... has no cj_vid on the row*").
- Full catalog convergence at 20/night $\approx$ **more than a year**, spent in an order unrelated to what customers want.

Rejected alternative — deferred ordering (accept the order, enrich later, place at CJ afterwards): it fails on two non-QPS problems. (1) *Variant selection* — a shallow goods has one optionless placeholder; after enrichment there may be 24 variants and nobody chose one. (2) *Price risk* — the placeholder carries the low bound of CJ's price range; charging before knowing the real variant cost can sell at a loss. Enriching **before** purchase fixes both. Deferred ordering remains possible later as a narrow single-variant-only path on the existing state machine.

2 · Solution — three parts, one principle

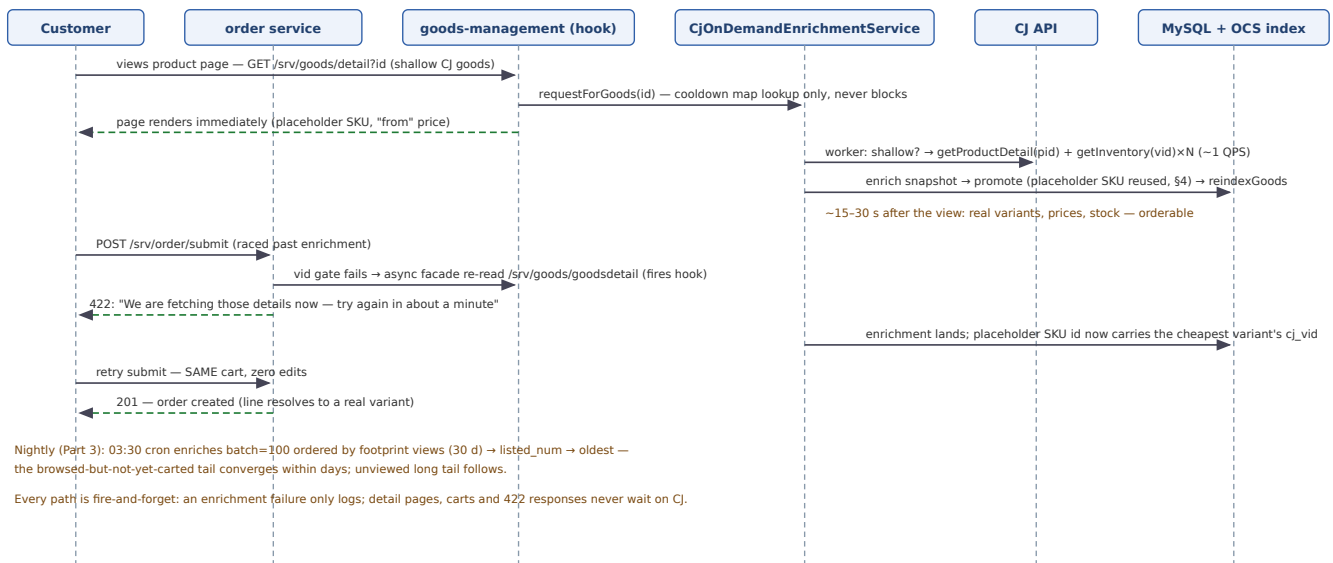
Principle: spend the CJ quota in demand order. One product's enrichment takes ~15–30 s of paced background work — far too slow for a synchronous checkout, but faster than a customer reads a product page.

	Trigger	Mechanism	Where
Part 1 Enrich-on-view	Customer opens a product detail page (or any consumer hits <code>/srv/goods/goodsdetail</code>)	<code>CjOnDemandEnrichmentService</code> : fire-and-forget enqueue → worker verifies the goods is a shallow CJ goods → <code>enrichByPid</code> (variants + prices + stock land, goods re-promoted + reindexed). Handles both promoted numeric ids and unpromoted <code>cj_<pid></code> ids (enrichment promotes those in the same pass).	goods-management
Part 2 Submit-block retry	The submit-time vid gate blocks a line	<code>CjOrderAvailabilityChecker</code> fires an async goods re-read through <code>LitellmGoodsFacade</code> — which lands on <code>/srv/goods/goodsdetail</code> and trips the Part-1 hook. The 422 message now says " <i>We are fetching those details now — please try again in about a minute.</i> " — and the retry genuinely works (see §4 self-heal).	order → goods-management
Part 3 Prioritized batch	Nightly cron 03:30	<code>selectForEnrichment</code> now orders by: never-enriched → customer views (footprints, 30 days) → CJ popularity (<code>listed_num</code>) → oldest-enriched. Batch default raised 20 → 100 .	litellm-db + goods-management

Guards (Part 1)

- **Cooldown** per product (default 6 h — aligned with the CJ raw-cache TTL): repeat views never re-spend quota (`on-demand-cooldown-seconds`).
- **Single worker thread**; the global CJ `RateLimiter` in `CJProductService` paces the actual API calls.
- **Bounded queue** (200): bursts drop with a log instead of backing up request threads; the nightly batch catches up. Read paths never wait — the hook costs one map lookup.
- **Kill switch**: `spring.cjdropship.on-demand-enrich-enabled: false` reverts to batch-only.

3 · Sequence



4 · Cart self-heal (placeholder SKU reuse)

The retry promise only works if the cart line survives enrichment. Previously the SKU diff (`upsertProducts` , matched-by- `cj_vid`) soft-deleted the vid-less placeholder when real variants landed — orphaning every cart line that pointed at it. Now, when a goods has exactly one vid-less placeholder and real variants arrive, the **cheapest variant inherits the placeholder's row id** (updated in place, creation time kept). The placeholder carried the "from" price — i.e. the cheapest variant's price — so the cart line's snapshot stays consistent, and the blocked customer's retry resolves to a real, orderable variant with no cart surgery. Multi-variant products effectively default to the entry variant; the customer can still re-select options on the (now fully variant-aware) product page.

5 · Files changed

File	Change
goods/application/search/CjOnDemandEnrichmentService.java	NEW — demand queue: cooldown map, bounded single worker, shallow-check, <code>requestForGoods</code> / <code>requestForPid</code>
goods/interfaces/rest/LitemallGoodsController.java	Hooks on GET /srv/goods/detail (numeric + <code>cj_<pid></code> branches) and GET /srv/goods/goodsdetail
goods/.../CJDropshippingConfig.java	on-demand-enrich-enabled: true · on-demand-cooldown-seconds: 21600 · on-demand-queue-capacity: 200 · enrich-batch-size: 20 → 100
goods/application/search/CjProductPromotionService.java	SKU diff: placeholder-reuse (cart self-heal, \$4)
litemall-db .../LitemallCjProductMapper.xml	<code>selectForEnrichment</code> : demand-prioritized ordering (footprints 30 d → <code>listed_num</code> → oldest)
order/application/internal/cj/CjOrderAvailabilityChecker.java	Blocked line → async facade re-read (<code>triggerOnDemandEnrichment</code>) + retry-in-a-minute message

No new tables, no Flyway migration, no new endpoints, no SPA changes. Domain-never-imports-Feign holds (the checker uses the existing facade).

6 · Live verification (2026-07-13)

- **Part 1 — enrich-on-view:** shallow goods 10000008 (2 vid-less SKUs) → one customer GET /srv/goods/detail through the gateway → ~40 s later: 6 real variants with distinct prices (25.63-52.13) and live stock, promoted + reindexed on the cj-on-demand-enrich worker. Log: CJ targeted enrichment: pid=... enriched → native goods 10000008.
- **Part 2 — submit-block chain:** a cart holding shallow goods 10001530 (earrings) → submit → friendly 422 naming the item → the checker's trigger enriched it (5 real variants) → order placed on retry (order 76).
- **Part 2+4 — automatic self-heal e2e (definitive):** cart holding placeholder SKU 275 of shallow goods 10000010 → submit #1 → 422 "We are fetching those details now..." → the blocked line triggered enrichment (~6 s, CJ cache warm) → BOTH placeholder rows healed in place (275 → cheapest variant 55.30, 12281 → next-cheapest 63.94; two new rows for the rest) → submit #2 with the **untouched cart** succeeded: order 77 created, its line referencing the original placeholder id 275.
- **Part 3 — prioritized batch:** selectForEnrichment verified against the live schema (footprint join + listed_num); batch default now 100.

7 · Operations

Concern	Answer
Quota worst case	Queue cap $200 \times (\sim 11 \text{ calls avg})$ per burst window, plus nightly $100 \times \sim 11$ — all serialized behind the global 1 QPS limiter. Raise/lower via <code>on-demand-queue-capacity</code> and <code>enrich-batch-size</code> .
Turn it off	<code>spring.cjdropship.on-demand-enrich-enabled: false</code> (batch-only), or <code>spring.cjdropship.enabled: false</code> (all CJ off).
Bot traffic on detail pages	Cooldown makes each product cost at most one enrichment per 6 h regardless of view count; queue cap bounds fan-out. Crawlers effectively pre-warm the popular catalog.
Observability	INFO CJ on-demand enrichment: shallow goods ... — enriching; WARN on failures and queue-full drops; convergence = shrinking count of <code>SELECT COUNT(*) ... NOT EXISTS(vid'd SKU)</code> .
Interaction with erosion tripwires	Orthogonal and compatible: enrichment only ever adds/updates variants; the prune/reconcile tripwires (fixed earlier today) keep absence-based deletion capped.